

Python Cheat Sheet

26 tables · 362 concepts · Updated 2026-05-28 · cheatgrid.com

Python is a **high-level, interpreted programming language** created by Guido van Rossum in 1991, emphasizing code readability through significant whitespace and a philosophy of "there should be one obvious way to do it." It's widely used across **web development, data science, automation, machine learning, and systems programming**, powered by a vast ecosystem of packages and an active global community. Python 3.14 (released October 2025) introduced **deferred annotation evaluation, t-strings**, and officially supported **free-threading** — marking one of the most significant runtime shifts in Python's history. The key to mastering Python isn't just knowing syntax — it's understanding when to use **list comprehensions over loops**, how **duck typing** works, and why context managers matter for resource safety.

CONTENTS

Data Types	2
Operators	2
String Operations	3
Slicing and Indexing	4
Control Flow	5
Functions	6
Comprehensions and Generators	6
Classes and OOP	7
Magic Methods (Dunder Methods)	7
Exception Handling	8
File Operations	9
Modules and Imports	10
Common Built-in Functions	10
Itertools Module	11
Decorators	12
Context Managers	13
Async/Await (Asyncio)	13
Type Hints	14
Regular Expressions	15
Dataclasses and Modern Class Features	15
Enum Module	16
Collections Module	17
Concurrency	17
Logging Module	18
Virtual Environments and Packaging	19
Advanced Patterns and Best Practices	20
References	21

TABLE 1

Data Types

Python's built-in types are the foundation every program builds on. Knowing which are mutable, which are hashable, and how each behaves in memory keeps you out of the most common Python traps — like assuming `bool` is *not* an `int`, or that two distinct lists with equal contents are the same object.

Type	Example	Description
int	<code>x = 42</code>	Arbitrary-precision integers — no overflow limit unlike most languages.
float	<code>y = 3.14</code>	- Double-precision floating-point (64-bit IEEE 754) beware rounding errors with exact comparisons.
str	<code>s = "hello"</code>	Immutable sequence of Unicode characters - supports f-strings, slicing, and a rich method surface.
bool	<code>flag = True</code>	Subclass of int — <code>True</code> is 1 and <code>False</code> is 0 - this surprises most newcomers
list	<code>arr = [1, 2, 3]</code>	Mutable ordered sequence — the most versatile and commonly used collection type.
dict	<code>d = {"key": "val"}</code>	- Mutable mapping of unique keys to values insertion-ordered since Python 3.7.
tuple	<code>t = (1, 2, 3)</code>	Immutable ordered sequence - hashable when elements are hashable — usable as dict keys.
set	<code>s = {1, 2, 3}</code>	- Mutable unordered collection of unique hashable elements - O(1) membership test.
frozenset	<code>fs = frozenset([1, 2])</code>	Immutable version of <code>set</code> - hashable — can be a dict key or set element.
None	<code>x = None</code>	Singleton representing absence of value - distinct from empty collections or zero
bytes	<code>b = b"data"</code>	Immutable sequence of byte values (0–255) - for binary data and network protocols
bytearray	<code>ba = bytearray(b"x")</code>	Mutable version of <code>bytes</code> — for in-place modification of binary data.
range	<code>r = range(10)</code>	Immutable sequence representing an arithmetic progression — memory-efficient for iteration.
complex	<code>z = 1 + 2j</code>	- Numbers with real and imaginary parts <code>j</code> suffix denotes the imaginary unit

TABLE 2

Operators

Every Python operator looks familiar from other languages, but most carry one or two Python-specific subtleties — true vs floor division, modulo whose sign follows the divisor, right-associative `**`, short-circuit `and` / `or` that return operands instead of booleans. The newer additions (`:=` walrus, `|` for dicts) reflect Python's continuing language design.

Operator	Example	Description
Arithmetic	<code>x + y</code> , <code>x - y</code> , <code>x * y</code>	Addition, subtraction, multiplication — work on numbers and some sequences (like <code>str</code> , <code>list</code>).
Division	<code>x / y</code>	Always returns float even for integer operands — true division since Python 3.
Floor division	<code>x // y</code>	- Rounds down to nearest integer - result type follows operands (<code>int//int → int</code>).
Modulo	<code>x % y</code>	Remainder — sign matches divisor , unlike C/Java where it matches the dividend.

Operator	Example	Description
Exponentiation	<code>x ** y</code>	- Raises <code>x</code> to power <code>y</code> right-associative and supports fractional/negative exponents
Comparison	<code>x == y</code> , <code>x < y</code> , <code>x >= y</code>	Equality and ordering — chainable like <code>1 < x < 10</code> (evaluates each pair once).
Identity	<code>x is y</code>	Tests if objects are the same instance in memory — not value equality.
Membership	<code>x in y</code>	Checks if <code>x</code> exists in container <code>y</code> $O(1)$ for sets and dicts
Logical AND	<code>x and y</code>	Returns first falsy value or last value if all truthy — short-circuits .
Logical OR	<code>x or y</code>	Returns first truthy value or last value if all falsy — short-circuits.
Logical NOT	<code>not x</code>	Boolean negation — returns <code>True</code> if <code>x</code> is falsy.
Bitwise AND	<code>x & y</code>	- Bitwise AND of integers - also set intersection for sets
Bitwise OR	<code>x y</code>	- Bitwise OR of integers - also set union for sets
Bitwise XOR	<code>x ^ y</code>	- Bitwise exclusive OR - also symmetric difference for sets
Bitwise NOT	<code>~x</code>	One's complement — inverts all bits - result is $-(x+1)$.
Left shift	<code>x << n</code>	- Shifts bits left by <code>n</code> - equivalent to multiplying by <code>2**n</code> .
Right shift	<code>x >> n</code>	- Shifts bits right by <code>n</code> - equivalent to floor-dividing by <code>2**n</code> .
Walrus operator	<code>(x := 5)</code>	Assigns and returns value in a single expression — useful in <code>while</code> and comprehensions (3.8+).
Dict merge operator	<code>d1 d2</code>	- Merges two dicts into a new dict <code>
• =</code> updates in-place (Python 3.9+).

TABLE 3

String Operations

Strings are immutable Unicode sequences with a rich method surface for searching, splitting, joining, and formatting. Modern code reaches for `f-strings` for interpolation, `str.join` for performant concatenation, and the `removeprefix` / `removesuffix` pair for safe edge trimming — and always passes `encoding="utf-8"` to `open()` to avoid locale-dependent surprises.

Method	Example	Description
f-strings	<code>f"{x} + {y} = {x+y}"</code>	Fastest and most readable formatting - supports expressions, format specs, and nested quotes since 3.12.
format()	<code>"{} {}".format(a, b)</code>	String formatting with positional/keyword arguments — more verbose than f-strings.
split()	<code>s.split(",")</code>	- Splits into list by delimiter default splits on any whitespace and removes empty strings
join()	<code>", ".join(items)</code>	Concatenates an iterable into a single string — preferred over += in loops for performance.
strip()	<code>s.strip()</code>	- Removes leading and trailing whitespace <code>lstrip()</code> / <code>rstrip()</code> for one side only
removeprefix() / removesuffix()	<code>s.removeprefix("pre")</code>	- Removes exact prefix or suffix if present - returns unchanged string otherwise (Python 3.9+).

Method	Example	Description
replace()	<code>s.replace("old", "new")</code>	- Returns copy with all occurrences replaced - optional <code>count</code> parameter to limit replacements
upper() / lower()	<code>s.upper()</code>	- Case conversion <code>casefold()</code> for aggressive case-insensitive comparisons across locales
startswith() / endswith()	<code>s.startswith("pre")</code>	Checks prefix/suffix — accepts a tuple of multiple options.
find() / index()	<code>s.find("sub")</code>	- Finds substring position <code>find()</code> returns -1 if not found, <code>index()</code> raises <code>ValueError</code> .
count()	<code>s.count("x")</code>	Returns the number of non-overlapping occurrences of a substring.
partition()	<code>s.partition(":")</code>	- Splits into 3-tuple (<code>before</code>, <code>sep</code>, <code>after</code>) - always returns 3 parts even if separator not found
isdigit() / isalpha()	<code>s.isdigit()</code>	- Character-type checks - many variants: <code>isalnum()</code> , <code>isspace()</code> , <code>islower()</code> , <code>isupper()</code> .
encode() / decode()	<code>s.encode("utf-8")</code>	- Converts <code>str</code> → <code>bytes</code> via encoding <code>bytes.decode("utf-8")</code> reverses it
center() / ljust() / rjust()	<code>s.center(20, "*")</code>	Pads string to a given width with fill character - useful for formatting output
zfill()	<code>"42".zfill(5)</code>	Pads with leading zeros to specified width — <code>"00042"</code> .
title() / capitalize()	<code>s.title()</code>	<code>title()</code> capitalizes each word <code>capitalize()</code> only the first character of the whole string
translate()	<code>s.translate(str.maketrans("abc", "ABC"))</code>	Replaces characters per a mapping table — efficient for multi-character substitution.
t-strings	<code>t"Hello {name}"</code>	Template strings producing a <code>Template</code> object for custom processing and safe interpolation (Python 3.14+).
%-formatting (legacy)	<code>"%s %d" % (s, n)</code>	C-style formatting — replaced by f-strings but still seen in legacy code.

TABLE 4

Slicing and Indexing

Slice syntax `[start:stop:step]` is the unified way to index Python sequences — zero-based, half-open, and quietly forgiving of out-of-range indices. The same three components handle reversal (`[::-1]`), shallow copies (`[:]`), every-other-element selection (`[::2]`), and reverse walks, making slicing one of the highest-leverage pieces of syntax to master.

Pattern	Example	Description
Positive indexing	<code>s[0] , s[2]</code>	Zero-based indexing from start — first element is <code>[0]</code> .
Negative indexing	<code>s[-1] , s[-2]</code>	Indexes from end — -1 is last element , -2 is second-to-last.
Basic slice	<code>s[1:4]</code>	Elements from index 1 up to (but not including) 4 — half-open interval.
Omit start	<code>s[:3]</code>	From beginning to index 3 — equivalent to <code>s[0:3]</code> .
Omit end	<code>s[2:]</code>	From index 2 to end — includes the last element.
Full copy	<code>s[:]</code>	Creates a shallow copy of the entire sequence.
Step slicing	<code>s[::2]</code>	Every 2nd element — third parameter is step size .
Reverse	<code>s[::-1]</code>	Reverses sequence — negative step iterates backward.
Negative step slice	<code>s[4:1:-1]</code>	From index 4 down to (but not including) 1.

TABLE 5

Control Flow

Python's control-flow keywords stay deliberately small: `if` / `elif` / `else` driven by truthiness, two loop forms with the unusual `else` clause, and (since 3.10) structural `match` with rich pattern types. The `match` statement goes far beyond a switch — it destructures sequences, mappings, class instances, and supports OR patterns, guards, and wildcard captures in one readable block.

Statement	Example	Description
if	<pre>if x > 0: print("pos")</pre>	Conditional execution — tests truthiness of expression.
elif	<pre>elif x == 0: print("zero")</pre>	Chain multiple conditions — only the first true branch executes.
else	<pre>else: print("neg")</pre>	Executes when all preceding conditions are false .
Ternary expression	<code>x if cond else y</code>	Inline conditional — returns <code>x</code> if cond is true , else <code>y</code> .
for loop	<pre>for i in range(5): print(i)</pre>	Iterates over any iterable object — most common loop type.
while loop	<pre>while x > 0: x -= 1</pre>	- Repeats while condition is truthy - check happens before each iteration
break	<code>break</code>	Exits innermost enclosing loop immediately.
continue	<code>continue</code>	Skips to the next iteration of the innermost loop.
else on loops	<pre>for i in items: pass else: print("done")</pre>	Executes if loop completes without break — useful for search patterns.
pass	<code>pass</code>	Null operation — syntactic placeholder where a statement is required.
match-case (literal)	<pre>match x: case 1: print("one")</pre>	Structural pattern matching — compares against literal values (Python 3.10+).
match OR pattern	<code>case 1 2 3:</code>	Matches multiple values in one case arm using <code>
</code> (Python 3.10+).
match capture pattern	<code>case Point(x=x, y=y):</code>	Destructures class instances by attribute names into local variables (Python 3.10+).
match sequence pattern	<code>case [first, *rest]:</code>	Matches and unpacks sequences — <code>*rest</code> captures remaining items (Python 3.10+).
match mapping pattern	<code>case {"action": a}:</code>	Matches dicts by key presence — <code>**rest</code> captures remaining keys (Python 3.10+).
match guard	<code>case x if x > 0:</code>	Adds a condition to any pattern — branch only executes if guard is true (Python 3.10+).
match wildcard	<code>case _:</code>	Catch-all pattern that matches anything — like <code>default</code> in other languages (Python 3.10+).

TABLE 6

Functions

Functions are first-class values in Python — they can be passed, returned, default-parameterized, decorated, and unpacked from iterables. Two non-obvious facts dominate the rough edges: default arguments are evaluated *once at definition time* (the source of the mutable-default trap), and the `/` and `*` separators in signatures enforce positional-only and keyword-only calling conventions explicitly.

Technique	Example	Description
Function definition	<pre>def func(a, b): return a + b</pre>	- Creates a reusable code block <code>return</code> without value returns <code>None</code>.
Default arguments	<pre>def f(x, y=10):</pre>	- Parameters with default values — must come after required params - evaluated once at definition.
Keyword arguments	<pre>f(x=1, y=2)</pre>	Named parameters — allows any order at call site and improves call-site clarity.
Positional-only params	<pre>def f(a, b, /, c):</pre>	Parameters before <code>/</code> cannot be passed by keyword — enforces positional calling (Python 3.8+).
Keyword-only params	<pre>def f(a, *, b):</pre>	Parameters after bare <code>*</code> must be passed by keyword — prevents accidental positional misuse.
<code>*args</code>	<pre>def f(*args):</pre>	Variable positional arguments — collected as a tuple .
<code>**kwargs</code>	<pre>def f(**kwargs):</pre>	Variable keyword arguments — collected as a dict .
Lambda	<pre>lambda x: x * 2</pre>	Anonymous function — single expression only, no statements.
Docstring	<pre>"""Description."""</pre>	First string in function body — accessible via <code>__doc__</code> and used by <code>help()</code> .
Type hints	<pre>def f(x: int) -> int:</pre>	Optional annotations — not enforced at runtime but used by type checkers.
Unpacking in calls	<pre>f(*args, **kwargs)</pre>	Expands sequences and mappings into function arguments at call time.

TABLE 7

Comprehensions and Generators

List, dict, and set comprehensions build a collection from an iterable in one expression — typically faster than the equivalent `for` / `append` loop and easier to read. Generator expressions and `yield`-based functions use almost the same syntax but produce lazy iterators, which is what lets Python stream gigabytes without loading them into memory.

Technique	Example	Description
List comprehension	<pre>[x**2 for x in range(10)]</pre>	Creates a list from an iterable — faster and cleaner than for-loop append.
With condition	<pre>[x for x in nums if x > 0]</pre>	Filters elements — condition acts as an inline filter on the iterable.
Nested comprehension	<pre>[x+y for x in a for y in b]</pre>	Multiple <code>for</code> clauses — left-to-right nesting order (outer loop first).
Dict comprehension	<pre>{k: v**2 for k, v in d.items()}</pre>	Creates a dict from an iterable — key-value pairs with unique keys (later values win).
Set comprehension	<pre>{x**2 for x in nums}</pre>	Creates a set — automatically deduplicates results.
Generator expression	<pre>(x**2 for x in range(10))</pre>	- Lazy evaluation — memory-efficient for large sequences - not stored in memory
Generator function	<pre>def gen(): yield x</pre>	Function with <code>yield</code> — pauses and resumes execution on each <code>next()</code> call.
yield from	<pre>yield from iterable</pre>	Delegates to a subiterator — simplifies nested generators (Python 3.3+).

TABLE 8

Classes and OOP

Python's class model favors flexibility over enforcement — no private attributes, no compile-time type checks, no formal interfaces. What you do get is a predictable MRO via C3 linearization, descriptor-driven property access, real multiple inheritance, and `abc.ABC` for defining enforced interfaces without metaclass boilerplate.

Concept	Example	Description
Class definition	<pre>class Dog: pass</pre>	Creates a new type — naming convention is PascalCase .
init	<pre>def __init__(self, name):</pre>	Constructor — initializes instance when object is created.
Instance attributes	<pre>self.name = name</pre>	Per-object data — defined in <code>__init__</code> or other methods via <code>self</code> .
Class attributes	<pre>class Dog: species = "Canis"</pre>	Shared across all instances — defined at class level .
Methods	<pre>def bark(self):</pre>	Functions bound to instance — first param is <code>self</code> by convention.
Inheritance	<pre>class Puppy(Dog):</pre>	Derives from parent class — inherits attributes and methods .
super()	<pre>super().__init__(name)</pre>	Calls parent class method — handles multiple inheritance via MRO correctly.
@property	<pre>@property def age(self):</pre>	Getter method — accessed like an attribute but computed dynamically.
@property.setter	<pre>@age.setter def age(self, val):</pre>	Setter method — allows validation and side effects on attribute assignment.
@classmethod	<pre>@classmethod def factory(cls):</pre>	Method receives class, not instance — for alternative constructors.
@staticmethod	<pre>@staticmethod def util():</pre>	Regular function in class namespace — no implicit first parameter .
Multiple inheritance	<pre>class C(A, B):</pre>	Inherits from multiple classes — uses C3 MRO to resolve method order.
ABC / @abstractmethod	<pre>from abc import ABC, abstractmethod class Shape(ABC): @abstractmethod def area(self): ...</pre>	Enforces interface — subclasses cannot be instantiated until all abstract methods are implemented.
str / repr	<pre>def __repr__(self):</pre>	String representations — <code>__repr__</code> for debugging , <code>__str__</code> for users .
slots	<pre>__slots__ = ('x', 'y')</pre>	Declares fixed attributes — reduces memory and blocks <code>__dict__</code> creation.
new	<pre>def __new__(cls):</pre>	- Static method that creates the instance before <code>__init__</code> - needed for immutable subclasses
init_subclass	<pre>def __init_subclass__(cls):</pre>	Hook called when a class is subclassed — lightweight alternative to metaclasses (Python 3.6+).

TABLE 9

Magic Methods (Dunder Methods)

Dunder methods are how custom classes plug into Python's syntax — `len()`, `obj[key]`, `for x in obj`, `==`, `+`, `with`, even calling an instance like a function. Implementing the right small set turns a class from "useful" to "feels built-in," and the data model documents every hook the language consults.

Method	Example	Description
init	<pre>def __init__(self):</pre>	Instance initialization — called when object is created .
str	<pre>def __str__(self):</pre>	Informal string representation — used by <code>str()</code> and <code>print()</code> .

Method	Example	Description
repr	<code>def __repr__(self):</code>	- Official string representation — should be valid Python when possible - fallback for <code>__str__</code> .
len	<code>def __len__(self):</code>	- Returns length — called by <code>len()</code> must return non-negative int.
bool	<code>def __bool__(self):</code>	Truthiness check — called by <code>bool()</code> and in boolean contexts - falls back to <code>__len__</code> .
getitem	<code>def __getitem__(self, key):</code>	Indexing operator <code>[]</code> — enables subscript access .
setitem	<code>def __setitem__(self, k, v):</code>	Item assignment <code>[k] = v</code> — makes object a mutable container .
delitem	<code>def __delitem__(self, key):</code>	Deletion via <code>del obj[key]</code> — required for full mutable container protocol.
iter	<code>def __iter__(self):</code>	Returns an iterator — enables for-loop support and unpacking.
next	<code>def __next__(self):</code>	Gets next value — raises <code>StopIteration</code> when exhausted.
contains	<code>def __contains__(self, item):</code>	Membership test <code>in</code> — should return boolean .
enter / exit	<code>def __enter__(self):</code>	Context manager protocol — enables with statement .
add	<code>def __add__(self, other):</code>	- Addition operator <code>+</code> — return new object - don't modify <code>self</code> .
eq	<code>def __eq__(self, other):</code>	- Equality operator <code>==</code> — default compares identity - defining it disables <code>__hash__</code> by default
hash	<code>def __hash__(self):</code>	- Makes instance hashable — required to use as dict key or set element - must be consistent with <code>__eq__</code> .
lt / gt	<code>def __lt__(self, other):</code>	Comparison operators <code><</code> and <code>></code> — enables sorting .
call	<code>def __call__(self):</code>	Makes instance callable like a function — enables obj() syntax .
format	<code>def __format__(self, spec):</code>	Custom format spec support — called by <code>format()</code> and f-string <code>{obj:spec}</code> syntax.
class_getitem	<code>def __class_getitem__(cls, item):</code>	Enables generic alias syntax <code>MyClass[T]</code> — supports runtime type parameterization.

TABLE 10

Exception Handling

Exceptions are Python's primary error-signalling mechanism, with a four-clause structure (`try` / `except` / `else` / `finally`) and clean chaining via `raise ... from ...` . Python 3.11's `ExceptionGroup` and `except*` give the language a way to express *multiple concurrent failures* — essential for `asyncio.TaskGroup` and structured-concurrency designs.

Technique	Example	Description
try-except	<code>try: risky() except ValueError:</code>	Catches exceptions — specific types should precede general ones.
Multiple exceptions	<code>except (ValueError, KeyError):</code>	Catches multiple types in one clause — tuple of exception classes .
except with variable	<code>except ValueError as e:</code>	Captures the exception object — access message and attributes via <code>e</code> .
else clause	<code>else: print("success")</code>	Runs if no exception raised in the <code>try</code> block — before <code>finally</code> .
finally	<code>finally: cleanup()</code>	Always executes — even if exception raised or <code>return</code> encountered - for resource cleanup

Technique	Example	Description
raise	<code>raise ValueError("msg")</code>	Throws an exception — bare <code>raise</code> re-raises the current active exception.
Custom exceptions	<code>class MyError(Exception):</code>	Define new exception types — inherit from <code>Exception</code> or a more specific subclass.
Exception chaining	<code>raise NewError() from e</code>	Links exceptions — preserves original cause for debugging tracebacks.
ExceptionGroup	<code>raise ExceptionGroup("msg", [ValueError(), KeyError()])</code>	Groups multiple exceptions together — handled with <code>except*</code> (Python 3.11+).
except*	<code>except* ValueError as eg:</code>	Catches matching exceptions from an ExceptionGroup — multiple <code>except*</code> clauses can match (Python 3.11+).
Bare except (avoid)	<code>except:</code>	• Catches all exceptions including <code>KeyboardInterrupt</code> — dangerous, hides bugs • avoid

TABLE 11

File Operations

File I/O uses a unified `open()` API parameterized by mode (`r` / `w` / `a` / `b` / `+`) and is best paired with `with` to guarantee cleanup on every exit path. Modern code increasingly leans on `pathlib.Path` for path manipulation and one-call `read_text` / `write_text` shortcuts — and remembers to pass `encoding="utf-8"` to dodge locale-dependent surprises across operating systems.

Technique	Example	Description
with statement	<code>with open("f.txt") as f:</code>	Context manager — auto-closes file even if an exception occurs.
Iterate file	<code>for line in f:</code>	Memory-efficient line iteration — preferred over <code>readlines()</code> for large files.
read()	<code>content = f.read()</code>	Reads entire file as string — memory-intensive for large files.
readline()	<code>line = f.readline()</code>	Reads a single line — includes newline character.
readlines()	<code>lines = f.readlines()</code>	Returns list of all lines — newlines preserved in each element.
write()	<code>f.write("text")</code>	Writes string to file — doesn't add newline automatically.
writelines()	<code>f.writelines(lines)</code>	Writes a sequence of strings — no newlines added automatically.
Mode "r" (default)	<code>open("f.txt", "r")</code>	Read mode — raises <code>FileNotFoundError</code> if file does not exist.
Mode "w"	<code>open("f.txt", "w")</code>	Write mode — truncates existing file or creates new.
Mode "a"	<code>open("f.txt", "a")</code>	• Append mode — adds to end of existing file • creates if absent
Mode "b"	<code>open("f.bin", "rb")</code>	• Binary mode — for non-text files • returns <code>bytes</code> objects
Mode "x"	<code>open("f.txt", "x")</code>	Exclusive creation — raises <code>FileExistsError</code> if file already exists .
pathlib.Path	<code>Path("f.txt").read_text(encoding="utf-8")</code>	• Object-oriented path API — cleaner than <code>os.path</code> • <code>copy()</code> / <code>move()</code> added in 3.14.
open() for reading	<code>f = open("f.txt", encoding="utf-8")</code>	Always specify <code>encoding="utf-8"</code> to avoid locale-dependent surprises on Windows.

TABLE 12

Modules and Imports

Python's import system stitches code into namespaces using `sys.path` (search list), `sys.modules` (cache), and a finder/loader stack you can extend. Idiomatic code prefers explicit `import` over `from X import *`, uses relative imports inside packages, and guards script-only code behind `if __name__ == "__main__":` so the same file works as both a library and a script.

Technique	Example	Description
import module	<code>import math</code>	Imports entire module — access via <code>math.func()</code> .
from import	<code>from math import sqrt</code>	Imports specific names — directly accessible without module prefix.
import as	<code>import numpy as np</code>	Aliases module to a shorter name for convenience.
if name == "main"	<code>if __name__ == "__main__":</code>	Checks if script is run directly vs imported — guards executable code.
Relative imports	<code>from . import sibling</code>	Within packages — dot notation for current/parent package.
init.py	<code>pkg/__init__.py</code>	Makes directory a package — can initialize the package and control public API.
sys.path	<code>sys.path.append(dir)</code>	Module search paths — modify to add custom locations.
importlib	<code>importlib.import_module(name)</code>	Dynamic imports — import by string name at runtime.
from import * (avoid)	<code>from math import *</code>	- Imports all public names — discouraged - pollutes namespace and hides origins

TABLE 13

Common Built-in Functions

The built-ins show up in nearly every Python program — `len`, `range`, `enumerate`, `zip`, `map`, `filter`, `sum`, `sorted`, `min` / `max`, `all` / `any`, `isinstance`. Mastering their `key=` arguments, lazy-iterator returns, and edge cases (vacuous `all([])`, `zip` truncation, `input` always returning `str`) keeps day-to-day code short and correct.

Function	Example	Description
len()	<code>len([1, 2, 3])</code>	Returns length of sequence or collection — calls <code>__len__</code> .
range()	<code>range(start, stop, step)</code>	Generates arithmetic sequence — stop is exclusive .
enumerate()	<code>enumerate(items, start=0)</code>	Adds counter to iterable — returns (index, value) pairs.
zip()	<code>zip(list1, list2)</code>	- Pairs elements from iterables — stops at shortest input <code>strict=True</code> raises if lengths differ (3.10+).
map()	<code>map(func, iterable)</code>	Applies function to each element — returns lazy iterator .
filter()	<code>filter(predicate, items)</code>	Filters elements where predicate is truthy — lazy iterator.
sorted()	<code>sorted(items, key=func)</code>	- Returns new sorted list — doesn't modify original <code>reverse=True</code> for descending
sum()	<code>sum(numbers, start=0)</code>	Sums a numeric iterable — optional starting value .
min() / max()	<code>min(items, key=func)</code>	Returns smallest/largest element — optional key function .
all()	<code>all([True, True, False])</code>	Returns True if all elements truthy or iterable is empty (vacuously true).
any()	<code>any([False, False, True])</code>	Returns True if any element truthy .
reversed()	<code>reversed(seq)</code>	Returns a reverse iterator — doesn't modify original.
isinstance()	<code>isinstance(obj, type)</code>	- Checks object type — accepts tuple of types - works with inheritance

Function	Example	Description
type()	<code>type(obj)</code>	Returns object's exact type — for checking, prefer <code>isinstance()</code> .
getattr() / setattr() / hasattr()	<code>getattr(obj, "name", default)</code>	Dynamic attribute access — <code>getattr</code> supports a default , avoids <code>AttributeError</code> .
vars() / dir()	<code>vars(obj)</code>	• <code>vars()</code> returns <code>__dict__</code> • <code>dir()</code> lists all attributes and methods of an object
abs() / divmod()	<code>abs(-5)</code> , <code>divmod(17, 5)</code>	• <code>abs()</code> returns magnitude • <code>divmod(a, b)</code> returns (quotient, remainder) as a tuple
hex() / oct() / bin()	<code>hex(255)</code> → <code>"0xff"</code>	Converts integers to hexadecimal/octal/binary string representations.
callable()	<code>callable(func)</code>	Returns <code>True</code> if object appears callable (has <code>__call__</code>).
hash()	<code>hash("hello")</code>	• Returns integer hash of a hashable object • consistent within a process
id()	<code>id(obj)</code>	Returns object's memory address — guaranteed unique for its lifetime.
input()	<code>input("Enter: ")</code>	Reads a line from stdin — always returns string .
print()	<code>print(*values, sep=" ", end="\n")</code>	Writes to stdout — customizable separator and line ending .

TABLE 14

Itertools Module

`itertools` is Python's grab-bag of iterator-combining building blocks — composable, lazy, and implemented in C for speed. The module covers infinite generators (`count` , `cycle`), combinatorics (`combinations` , `permutations` , `product`), grouping by adjacent runs (`groupby`), windowing (`pairwise` , `batched`), and reduction (`accumulate`).

Function	Example	Description
chain()	<code>chain(it1, it2)</code>	Concatenates iterables — flattens multiple iterables into single iterator.
chain.from_iterable()	<code>chain.from_iterable(nested)</code>	Flattens one level of nesting lazily — like <code>chain(*nested)</code> without unpacking.
islice()	<code>islice(it, start, stop)</code>	Slices an iterator — doesn't support negative indices .
compress()	<code>compress('ABCDEF', [1,0,1,0,1,1])</code>	Filters data by a boolean selector iterable — yields A, C, E, F .
filterfalse()	<code>filterfalse(lambda x: x<5, [1,4,6,3,8])</code>	Yields elements where predicate is false — opposite of <code>filter()</code> .
starmap()	<code>starmap(pow, [(2,5), (3,2)])</code>	Applies function with arguments unpacked from each tuple in iterable.
pairwise()	<code>pairwise([1, 2, 3])</code>	Returns overlapping pairs of consecutive elements — (1,2), (2,3) (Python 3.10+).
batched()	<code>batched(range(10), 3)</code>	Splits iterable into fixed-size tuples — last batch may be shorter (Python 3.12+).
groupby()	<code>groupby(items, key=func)</code>	Groups consecutive elements by key — input must be sorted by same key first.
accumulate()	<code>accumulate(nums, func)</code>	• Running totals or reductions — default is addition • accepts any binary function
takewhile()	<code>takewhile(pred, it)</code>	Yields elements while predicate is true — stops at first false.
dropwhile()	<code>dropwhile(pred, it)</code>	Skips elements while predicate is true — then yields all remaining.

Function	Example	Description
<code>tee()</code>	<code>tee(iterable, n=2)</code>	Splits one iterator into <code>n</code> independent iterators — use before an iterator is consumed.
<code>zip_longest()</code>	<code>zip_longest(a, b, fillvalue=None)</code>	Like <code>zip()</code> but continues to the longest input, filling shorter ones with <code>fillvalue</code> .
<code>combinations()</code>	<code>combinations(items, r)</code>	All <code>r</code> -length combinations — order doesn't matter , no repeated elements.
<code>combinations_with_replacement()</code>	<code>combinations_with_replacement('AB', 2)</code>	All <code>r</code> -length combinations allowing repeated elements — yields <code>AA</code> , <code>AB</code> , <code>BB</code> .
<code>permutations()</code>	<code>permutations(items, r)</code>	All <code>r</code> -length permutations — order matters , no repeated elements.
<code>product()</code>	<code>product(a, b)</code>	Cartesian product — equivalent to nested for-loops .
<code>count()</code>	<code>count(start=0, step=1)</code>	Infinite counter — useful with <code>zip()</code> or <code>takewhile()</code> .
<code>cycle()</code>	<code>cycle([1, 2, 3])</code>	Repeats iterable infinitely — saves copy of all elements in memory.
<code>repeat()</code>	<code>repeat(10, times=3)</code>	Repeats object <code>n</code> times — infinite if times omitted .

TABLE 15

Decorators

Decorators wrap a function or class to add behavior without changing the callsite. The syntax `@decorator` is sugar for `func = decorator(func)` , extending naturally to parameterized decorators (three nesting levels) and to stacking multiple decorators (applied bottom-up). `functools.wraps` is mandatory in every wrapper to preserve the original's metadata.

Technique	Example	Description
Function decorator	<code>@decorator</code> <code>def func():</code>	Wraps function to modify behavior — sugar for <code>func = decorator(func)</code> .
Simple decorator	<code>def deco(f):</code> <code>def wrap():</code> <code>return f()</code> <code>return wrap</code>	Returns a wrapper function that replaces the original .
functools.wraps	<code>@wraps(f)</code> <code>def wrapper():</code>	Preserves function metadata (<code>__name__</code> , <code>__doc__</code> , <code>__wrapped__</code>) — use inside every wrapper.
Decorator with args	<code>@deco(arg)</code> <code>def func():</code>	Returns a decorator — three-level nesting : <code>deco(arg)(func)</code> .
Stacking decorators	<code>@deco1</code> <code>@deco2</code> <code>def f():</code>	Multiple decorators applied bottom-up : <code>deco1(deco2(f))</code> .
Class decorator	<code>@deco</code> <code>class MyClass:</code>	Modifies or replaces a class — callable receives and returns the class .
lru_cache	<code>@lru_cache(maxsize=128)</code>	Memoizes function results with LRU eviction — <code>cache_info()</code> reports hits and misses.
functools.cache	<code>@cache</code>	Unbounded memoization — faster than <code>lru_cache(None)</code> as no eviction overhead (Python 3.9+).
functools.cached_property	<code>@cached_property</code> <code>def value(self):</code>	Computes property once per instance and caches as a normal attribute — delete attribute to reset (Python 3.8+).
functools.total_ordering	<code>@total_ordering</code> <code>class C:</code>	Generates all comparison methods from just <code>__eq__</code> and one ordering method (<code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> , or <code>__ge__</code>).
Method decorator	<code>@staticmethod</code> <code>def method():</code>	Built-in decorators : <code>@classmethod</code> , <code>@staticmethod</code> , <code>@property</code> .

TABLE 16

Context Managers

Context managers — driven by `__enter__` / `__exit__` (or `__aenter__` / `__aexit__` for async) — guarantee deterministic resource cleanup *even if the block raises*. The `contextlib` module makes new managers trivial via the `@contextmanager` generator decorator and packages handy ones like `suppress`, `closing`, and the dynamic-composition `ExitStack`.

Technique	Example	Description
with statement	<code>with open("f") as f:</code>	Ensures resource cleanup — calls <code>__enter__</code> and <code>__exit__</code> .
Multiple contexts	<code>with open("a") as f1, open("b") as f2:</code>	Manages multiple resources — left-to-right acquisition , right-to-left release.
contextlib.contextmanager	<code>@contextmanager def ctx(): yield</code>	Creates a context manager from a generator — yield separates setup from teardown .
Custom enter / exit	<code>def __enter__(self): return self</code>	Full protocol implementation — <code>__exit__</code> receives <code>(exc_type, exc_val, exc_tb)</code> .
contextlib.suppress	<code>with suppress(FileNotFoundError):</code>	Ignores specified exceptions — cleaner than <code>try: ... except: pass</code> .
contextlib.closing	<code>with closing(obj):</code>	Calls <code>close()</code> on exit — for objects without a native context manager .
ExitStack	<code>with ExitStack() as stack:</code>	Manages a variable number of context managers dynamically.

TABLE 17

Async/Await (Asyncio)

`asyncio` delivers cooperative single-threaded concurrency: many waiting coroutines, one running at a time, no blocking. `await` is the only yield point — every awaited call must be non-blocking — and Python 3.11+ adds *structured concurrency* via `TaskGroup` plus deadline-based `timeout()`. The synchronization primitives (`Lock`, `Event`, `Semaphore`, `Queue`) mirror the `threading` module but are awaitable, not blocking.

Technique	Example	Description
async def	<code>async def fetch():</code>	Defines a coroutine function — must be awaited or scheduled as a task.
await	<code>result = await coro()</code>	Suspends coroutine — yields control to event loop until result is ready.
asyncio.run()	<code>asyncio.run(main())</code>	- Entry point — creates event loop and runs coroutine - closes loop on completion
asyncio.create_task()	<code>task = create_task(coro())</code>	Schedules coroutine to run concurrently with other tasks.
asyncio.TaskGroup	<code>async with TaskGroup() as tg: tg.create_task(coro())</code>	Structured concurrency — auto-cancels remaining tasks on failure (Python 3.11+).
asyncio.gather()	<code>await gather(coro1(), coro2())</code>	Runs multiple coroutines concurrently — waits for all to complete.
asyncio.timeout()	<code>async with asyncio.timeout(5):</code>	Context manager with deadline — raises <code>TimeoutError</code> if exceeded (Python 3.11+).
asyncio.wait_for()	<code>await wait_for(coro(), timeout=5)</code>	Adds timeout to a coroutine — raises TimeoutError if exceeded.
asyncio.Lock	<code>async with asyncio.Lock():</code>	Mutex lock for async tasks — prevents concurrent access to shared state.
asyncio.Event	<code>event = asyncio.Event() await event.wait()</code>	Notifies multiple tasks that something happened — <code>set()</code> , <code>clear()</code> , <code>wait()</code> .
asyncio.Semaphore	<code>async with asyncio.Semaphore(10):</code>	Limits number of concurrent coroutines accessing a resource.

Technique	Example	Description
asyncio.Queue	<pre>q = asyncio.Queue() await q.put(x) x = await q.get()</pre>	FIFO queue for producer-consumer patterns — <code>task_done()</code> + <code>join()</code> for completion tracking.
asyncio.sleep()	<pre>await asyncio.sleep(1)</pre>	Non-blocking sleep — yields control to other coroutines.
async for	<pre>async for item in async_iter:</pre>	Iterates over an async iterable — calls <code>__aiter__</code> and <code>__anext__</code> .
async with	<pre>async with resource:</pre>	Async context manager — awaits <code>__aenter__</code> and <code>__aexit__</code> .

TABLE 18

Type Hints

Type hints are runtime metadata that static checkers (`mypy`, `pyright`, `pyre`) consume to find bugs before they ship. PEP 585 (Python 3.9) made built-in generics work directly (`list[int]`), PEP 604 (3.10) introduced `int | None` unions, PEP 695 (3.12) added inline generic-parameter syntax, and PEP 649/749 (3.14) introduced **deferred evaluation** — annotations are no longer evaluated at import time, dramatically improving startup performance.

Technique	Example	Description
Basic types	<pre>x: int = 5</pre>	Annotates variable type — not enforced at runtime.
Function annotations	<pre>def f(x: int) -> str:</pre>	Parameter and return types — used by type checkers like <code>mypy</code> and <code>pyright</code> .
Built-in generics	<pre>items: list[int] d: dict[str, int]</pre>	Use built-in types directly as generics — no <code>List</code> / <code>Dict</code> imports needed (Python 3.9+).
Union (pipe syntax)	<pre>x: int str</pre>	• Multiple allowed types — modern <code> </code> syntax replaces <code>Union[int, str]</code> (Python 3.10+).
Optional[T]	<pre>x: Optional[int] = None</pre>	• Shorthand for <code>Union[T, None]</code> — prefer <code>T None</code> in new code
TypedDict	<pre>class Config(TypedDict): host: str port: int</pre>	Typed dict with specific key/value types — enables type checking on dict literals.
Callable	<pre>f: Callable[[int], str]</pre>	Function type — list of param types then return type.
Protocol	<pre>class Sized(Protocol): def __len__(self): ...</pre>	Structural subtyping — duck typing with static type checking.
Literal	<pre>x: Literal["a", "b"]</pre>	Restricts to specific values — useful for discriminated unions.
TypeVar	<pre>T = TypeVar('T')</pre>	Generic type variable — for parametric polymorphism in functions and classes.
Type parameter syntax	<pre>def f[T](x: T) -> T:</pre>	Inline generic type variables on functions and classes — replaces <code>TypeVar</code> (Python 3.12+).
type statement	<pre>type Vector = list[float]</pre>	Declares a type alias with lazy evaluation — replaces <code>TypeAlias</code> annotation (Python 3.12+).
ParamSpec	<pre>P = ParamSpec('P')</pre>	Captures the parameter types of a callable — essential for typing decorators that preserve signatures.
Unpack	<pre>def f(**kwargs: Unpack[Config]):</pre>	Types <code>`kwargs`</code> using a <code>TypedDict`</code> — enables per-key type checking (Python 3.12+).
Any	<pre>x: Any</pre>	• Disables type checking — avoid when possible • escape hatch
Typels	<pre>def is_str(x) -> TypeIs[str]:</pre>	Narrows type in both if/else branches — safer than <code>TypeGuard</code> (Python 3.13+).
@override	<pre>@override def method(self):</pre>	Marks method as overriding parent — type checkers flag mismatches (Python 3.12+).

Technique	Example	Description
Deferred annotations	<code>def f(x: MyClass) -> None:</code>	In Python 3.14+ annotations are not evaluated at import time — forward references work natively without quotes.

TABLE 19

Regular Expressions

The `re` module exposes a single regex engine through both functional (`re.search` , `re.findall` , `re.sub`) and compiled (`re.compile(...)`) APIs. The biggest pitfalls are `re.match` vs `re.search` (anchored vs unanchored) and forgetting `r"..."` raw strings — without them, Python's own backslash processing eats half your pattern before the engine sees it.

Function	Example	Description
re.search()	<code>re.search(r"\d+", s)</code>	Searches anywhere in string — returns first match object or <code>None</code> .
re.match()	<code>re.match(r"\d+", s)</code>	Matches at start of string only — returns match object or <code>None</code> .
re.fullmatch()	<code>re.fullmatch(r"\d+", s)</code>	Matches entire string — both <code>^</code> and <code>\$</code> are implicit.
re.findall()	<code>re.findall(r"\d+", s)</code>	Returns list of all matches — non-overlapping strings.
re.finditer()	<code>re.finditer(r"\d+", s)</code>	Returns iterator of match objects — memory-efficient for large inputs.
re.sub()	<code>re.sub(r"\d+", "X", s)</code>	Replaces matches — accepts function as replacement for dynamic substitutions.
re.split()	<code>re.split(r"\s+", s)</code>	Splits by pattern — preserves groups in result if pattern has groups.
re.compile()	<code>p = re.compile(r"\d+")</code>	Precompiles pattern — faster for reuse across many strings.
Groups	<code>m.group(1)</code>	Captures subpatterns — <code>group(0)</code> is entire match , <code>group(1)</code> first capture.
Named groups	<code>(?P<name>\d+)</code>	Named capture — access via <code>m.group('name')</code> or <code>m.groupdict()</code> .
Non-greedy	<code>r"\d+?"</code>	<code>?</code> after quantifier — matches as few characters as possible .
Lookahead / lookbehind	<code>r"\d+(?= px)"</code>	Zero-width assertions — <code>(?=...)</code> lookahead, <code>(?<=...)</code> lookbehind — don't consume chars .
re.IGNORECASE	<code>re.search(r"a", s, re.I)</code>	• Case-insensitive matching — combine flags with <code>re.I</code> <code>re.M</code> .
re.DOTALL	<code>re.search(r".+", s, re.S)</code>	Makes <code>.</code> match newlines too — useful for multi-line text blocks.
re.MULTILINE	<code>re.search(r"^word", s, re.M)</code>	<code>^</code> and <code>\$</code> match start/end of each line , not just the string.
Raw strings	<code>r"\d+"</code>	Prefix with <code>r</code> — avoids double-escaping backslashes in patterns.

TABLE 20

Dataclasses and Modern Class Features

`@dataclass` (Python 3.7+) generates `__init__` , `__repr__` , and `__eq__` from type-annotated attributes, with knobs for `frozen` , `slots` , and `kw_only` to fit specific use cases. For runtime validation and coercion at system boundaries, the ecosystem leans on `pydantic` ; for richer feature sets without the validation cost, on `attrs` .

Technique	Example	Description
@dataclass	<code>@dataclass class Point: x: float y: float</code>	Auto-generates <code>__init__</code> , <code>__repr__</code> , <code>__eq__</code> — less boilerplate (Python 3.7+).
Field with default	<code>x: int = 0</code>	Default values — mutable defaults must use <code>field(default_factory=...)</code> .

Technique	Example	Description
field()	<code>items: list = field(default_factory=list)</code>	Customizes field behavior — prevents shared mutable defaults across instances.
frozen=True	<code>@dataclass(frozen=True)</code>	Makes instances immutable — enables hashing for use as dict keys.
slots=True	<code>@dataclass(slots=True)</code>	Generates <code>__slots__</code> for reduced memory — prevents dynamic attribute creation (Python 3.10+).
kw_only=True	<code>@dataclass(kw_only=True)</code>	Forces keyword-only arguments in <code>__init__</code> — prevents positional mistakes (Python 3.10+).
post_init	<code>def __post_init__(self):</code>	Runs after <code>__init__</code> — for computed fields or validation.
InitVar	<code>temp: InitVar[int]</code>	Init-only variable — not stored as field , available in <code>__post_init__</code> .
asdict() / astuple()	<code>asdict(obj)</code>	Converts dataclass to dict/tuple — deep-copies nested dataclasses.
attrs	<code>@define class C:</code>	Third-party alternative — more features than dataclasses (validators, converters, slots by default).
Pydantic	<code>class User(BaseModel):</code>	Data validation library — runtime type checking , coercion, and JSON schema generation.

TABLE 21

Enum Module

The `enum` module provides a way to define named constant sets with distinct identity, preventing typos and magic-string comparisons. Python 3.11 added `StrEnum` and `IntEnum` auto-values that produce the lowercased member name, and Python 3.12 improved the `repr` of enum members and simplified subclassing.

Type	Example	Description
Enum	<code>class Color(Enum): RED = 1 GREEN = 2</code>	Base class for enumerations — members have unique identity and are accessed as <code>Color.RED</code> .
auto()	<code>RED = auto()</code>	Automatically assigns values — default is sequential integers starting at 1.
IntEnum	<code>class Status(IntEnum): OK = 200</code>	Enum whose members are also <code>int</code> — compatible with integer APIs.
StrEnum	<code>class Direction(StrEnum): NORTH = auto()</code>	Members are also <code>str</code> — <code>auto()</code> generates the lowercased member name (Python 3.11+).
Flag	<code>class Perm(Flag): READ = auto() WRITE = auto()</code>	• Members support bitwise operations — combine with <code>
</code> , test with <code>in</code> .
IntFlag	<code>class Mode(IntFlag): R = 4 W = 2</code>	Flag + int compatibility — members behave as integers for system calls and bitmasks.
Enum iteration	<code>list(Color)</code>	Enumerates all members in definition order.
Enum lookup	<code>Color(1) , Color["RED"]</code>	Access by value (<code>Color(1)</code>) or by name (<code>Color["RED"]</code>).
member.value / .name	<code>Color.RED.value , Color.RED.name</code>	Access the raw value or the member name string .

TABLE 22

Collections Module

The `collections` module provides specialized container datatypes that extend Python's built-in `dict`, `list`, `set`, and `tuple`. `deque` is the go-to for $O(1)$ queue operations, `OrderedDict` retains its usefulness for LRU implementations despite regular dicts being ordered since 3.7, and `Counter` eliminates a common tally-building loop pattern entirely.

Type	Example	Description
deque	<pre>from collections import deque dq = deque([1,2,3], maxlen=5)</pre>	- Double-ended queue — $O(1)$ append and pop on both ends <code>list.pop(0)</code> is $O(n)$.
deque.appendleft() / popleft()	<pre>dq.appendleft(0) dq.popleft()</pre>	Efficient left-side operations — core advantage over <code>list</code> for queue usage.
deque with maxlen	<pre>deque(maxlen=10)</pre>	Bounded deque — oldest items auto-discarded when full - perfect for sliding windows
defaultdict	<pre>defaultdict(list)</pre>	Dict that auto-creates missing keys using a factory function.
Counter	<pre>Counter("abracadabra")</pre>	Dict subclass for counting hashable objects — <code>most_common(n)</code> returns top-n items.
Counter arithmetic	<pre>c1 + c2 , c1 - c2 , c1 & c2</pre>	Supports <code>+</code>, <code>-</code>, <code>&</code> (intersection), <code> </code> (union) — results keep only positive counts.
Counter.total()	<pre>c.total()</pre>	Returns sum of all counts (Python 3.10+).
namedtuple	<pre>Point = namedtuple('Point', 'x y')</pre>	Tuple with named fields — immutable, memory-efficient alternative to a class.
OrderedDict	<pre>OrderedDict()</pre>	- Dict that remembers insertion order and supports <code>move_to_end()</code> - still useful for LRU caches
OrderedDict.move_to_end()	<pre>od.move_to_end("key")</pre>	Moves key to end (or beginning with <code>last=False</code>) — key primitive for LRU cache implementation.
ChainMap	<pre>ChainMap(local, global_, builtins)</pre>	- Groups multiple dicts — searches sequentially without copying - write goes to first map
UserDict / UserList	<pre>class MyDict(UserDict):</pre>	Subclass-friendly wrappers around <code>dict</code> / <code>list</code> — safer to subclass than the built-ins directly.

TABLE 23

Concurrency

Python offers three concurrency models: **threading** (shared memory, GIL-limited for CPU work, good for I/O), **multiprocessing** (separate processes, true parallelism, higher overhead), and **asyncio** (cooperative single-thread, ultra-low overhead, requires async-aware code). `concurrent.futures` wraps both threading and multiprocessing behind one uniform executor interface; Python 3.14 adds `InterpreterPoolExecutor` for sub-interpreter concurrency without the IPC overhead of full processes.

Technique	Example	Description
ThreadPoolExecutor	<pre>with ThreadPoolExecutor(max_workers=4) as ex: fut = ex.submit(func, arg)</pre>	- Thread pool — best for I/O-bound tasks - GIL limits CPU parallelism
ProcessPoolExecutor	<pre>with ProcessPoolExecutor() as ex: fut = ex.submit(func, arg)</pre>	- Process pool — true CPU parallelism - data is pickled between processes
executor.map()	<pre>list(ex.map(func, items))</pre>	Parallel map — returns results in submission order , raises first exception encountered.

Technique	Example	Description
Future.result()	<code>fut.result(timeout=5)</code>	Blocks until result available — re-raises any exception thrown by the worker.
as_completed()	<code>for f in as_completed(futures):</code>	Yields futures as they complete — better throughput than waiting in submission order.
InterpreterPoolExecutor	<code>with InterpreterPoolExecutor() as ex:</code>	Runs tasks in sub-interpreters — true parallelism without pickle overhead (Python 3.14+).
threading.Thread	<code>t = Thread(target=f, args=(1,))</code> <code>t.start(); t.join()</code>	Spawns a native OS thread — use <code>daemon=True</code> to die with main thread.
threading.Lock	<code>lock = Lock()</code> <code>with lock:</code>	Mutex for shared state — <code>with lock:</code> is preferred (auto-releases on exceptions).
threading.Event	<code>e = Event()</code> <code>e.set(); e.wait()</code>	Allows threads to signal each other — <code>wait()</code> blocks until <code>set()</code> .
threading.Semaphore	<code>sem = Semaphore(3)</code>	Limits concurrent access to <code>n</code> threads — <code>acquire()</code> decrements, <code>release()</code> increments.
multiprocessing.Pool	<code>with Pool() as pool:</code> <code>pool.map(func, items)</code>	Higher-level multiprocessing API — automatically manages worker process lifecycle.
multiprocessing.Queue	<code>q = mp.Queue()</code> <code>q.put(x); q.get()</code>	Process-safe queue — IPC via pickle across process boundaries.
GIL and free-threading	<code>python3.14 -X gil=0</code>	Python 3.14 makes free-threading production-ready (PEP 779) — true thread parallelism possible.

TABLE 24

Logging Module

Python's `logging` is a hierarchical system: every logger is a node in a tree rooted at the root logger, and events propagate upward unless `propagate=False`. Application code should always call `logging.getLogger(__name__)`, let libraries configure nothing themselves, and leave root-level configuration to the application entry point.

Technique	Example	Description
basicConfig()	<code>logging.basicConfig(level=logging.INFO,</code> <code>format="%asctime)s %(levelname)s</code> <code>%(message)s")</code>	- Configures root logger — one-time call for simple scripts - must be before first log call
getLogger()	<code>logger = logging.getLogger(__name__)</code>	- Gets or creates a named logger — hierarchy via dot notation - always use <code>__name__</code>.
Log levels	<code>logger.debug("...")</code> <code>logger.info("...")</code> <code>logger.warning("...")</code> <code>logger.error("...")</code> <code>logger.critical("...")</code>	Five levels: DEBUG < INFO < WARNING < ERROR < CRITICAL — only events at or above set level are emitted.
StreamHandler	<code>h = logging.StreamHandler()</code> <code>logger.addHandler(h)</code>	Outputs to stderr (default) or any stream — standard handler for console output.
FileHandler	<code>h = logging.FileHandler("app.log")</code>	Writes to a file — use <code>RotatingFileHandler</code> for size-limited rotation.
RotatingFileHandler	<code>RotatingFileHandler("app.log",</code> <code>maxBytes=1e6, backupCount=3)</code>	Rotates log file when it reaches <code>maxBytes</code> — keeps <code>backupCount</code> backups.
Formatter	<code>fmt = Formatter("%(asctime)s - %(name)s -</code> <code>%(message)s")</code> <code>h.setFormatter(fmt)</code>	Controls log record format — standard attributes include <code>asctime</code> , <code>name</code> , <code>levelname</code> , <code>message</code> .

Technique	Example	Description
logger.exception()	<code>except Exception: logger.exception("Error")</code>	Logs at ERROR level plus the full stack trace — use inside <code>except</code> blocks.
Propagate	<code>logger.propagate = False</code>	Stops log records bubbling up to parent loggers — avoids duplicate output.
dictConfig / fileConfig	<code>logging.config.dictConfig(cfg)</code>	Configures logging from a dict or file — preferred for production over <code>basicConfig</code> .
LoggerAdapter	<code>adapter = LoggerAdapter(logger, { "req_id": id })</code>	Adds contextual data to every log call without modifying the message format.

TABLE 25

Virtual Environments and Packaging

Isolation is the first principle: one project, one environment, so dependency versions never interfere. `venv` is the built-in option (zero install, always available), while `uv` (Astral) is a fast Rust-based alternative that manages both environments *and* Python installations with near-instant dependency resolution.

Tool	Example	Description
venv create	<code>python -m venv .venv</code>	Creates a virtual environment — installs pip + copies/symlinks interpreter.
venv activate	<code>source .venv/bin/activate .venv\Scripts\activate</code>	Activates environment — modifies PATH so <code>python</code> / <code>pip</code> point to venv.
pip install	<code>pip install requests pip install -r requirements.txt</code>	Installs packages — <code>-r</code> installs from requirements file.
pip freeze	<code>pip freeze > requirements.txt</code>	Snapshots current installed packages with pinned versions.
uv init	<code>uv init example</code>	Creates a new project with <code>pyproject.toml</code> — fast Rust-based tool (Astral).
uv run	<code>uv run script.py</code>	Runs script in an isolated, auto-created environment — no manual activation needed.
uv add	<code>uv add requests</code>	Adds dependency to <code>pyproject.toml</code> and updates lockfile atomically.
uv sync	<code>uv sync</code>	Installs dependencies from lockfile — reproduces exact environment.
uv python install	<code>uv python install 3.12</code>	Downloads and installs a specific Python version — managed alongside venv.
pyproject.toml	<code>[project] name = "myapp" requires-python = ">=3.11"</code>	• Modern packaging standard — replaces <code>setup.py</code> • supported by pip, uv, poetry, flit
pyenv	<code>pyenv install 3.11.5 pyenv local 3.11.5</code>	Manages multiple Python versions side-by-side — sets <code>.python-version</code> file per directory.
poetry	<code>poetry new project poetry add requests</code>	Package manager with lockfile and publishing support — <code>poetry install</code> reproduces env.

TABLE 26

Advanced Patterns and Best Practices

These patterns represent idiomatic Python thinking: prefer iteration over indexing, exploit descriptors and `__slots__` for performance-critical classes, compose with `functools.partial` (and Python 3.14's `Placeholder` for gap filling), and exploit generators for lazy pipelines that process data without materializing everything in memory.

Technique	Example	Description
Generator pipeline	<pre>evens = (x for x in data if x%2==0) result = sum(evens)</pre>	- Chains generators lazily — never materializes intermediate lists - memory-efficient for large datasets
functools.partial	<pre>double = partial(mul, 2)</pre>	Pre-fills arguments — creates specialized callable from a general function.
functools.Placeholder	<pre>f = partial(pow, ..., 2)</pre>	Marks positional gaps in <code>partial()</code> that callers must fill — <code>...</code> is the Placeholder (Python 3.14+).
functools.reduce	<pre>reduce(lambda a,b: a+b, nums)</pre>	Folds a sequence left-to-right into a single value.
Descriptor protocol	<pre>def __get__(self, obj, type):</pre>	Implements per-attribute access — foundation for <code>property</code> , <code>classmethod</code> , <code>staticmethod</code> .
slots	<pre>__slots__ = ("x", "y")</pre>	Replaces per-instance <code>__dict__</code> with fixed memory layout — reduces RAM and speeds attribute access.
Walrus operator	<pre>while chunk := f.read(8192):</pre>	- Assignment expression — assigns and evaluates in one step - avoids repeated calls
Positional-only params	<pre>def f(a, b, /, c):</pre>	<code>/</code> separator — <code>a</code>, <code>b</code> can only be passed by position - prevents API-breaking renames
Keyword-only params	<pre>def f(*, a, b):</pre>	<code>*</code> separator — <code>a</code> , <code>b</code> must be passed by keyword .
Metaclass	<pre>class Meta(type): def __new__(mcs, ...):</pre>	Controls class creation — used by ORMs, ABCs, and plugin registries.
init_subclass	<pre>def __init_subclass__(cls, **kw):</pre>	Hook called when a class is subclass ed — simpler plugin registration than metaclasses.
Singleton pattern	<pre>def __new__(cls): if not cls._instance: cls._instance = super().__new__(cls) return cls._instance</pre>	Ensures one instance via <code>__new__</code> — thread-safe version requires a lock.
t-strings (PEP 750)	<pre>tpl = t"Hello {name}"</pre>	- Like f-strings but deferred — returns a <code>Template</code> object instead of a string - enables safe HTML/SQL generation (Python 3.14+).
deferred annotations (PEP 749)	<pre>def f() -> MyClass:</pre>	- Annotations are not evaluated at import time in Python 3.14+ — forward references require no quotes - use <code>annotationlib</code> to evaluate explicitly
concurrent.interpreters	<pre>import concurrent.interpreters</pre>	New in Python 3.14 (PEP 734) — runs code in isolated sub-interpreters with true parallelism via <code>InterpreterPoolExecutor</code> .

References

Official Documentation

1. Python 3 Official Documentation
3. Python 3.13 What's New
5. Python 3.11 What's New
7. Python Built-in Types
9. Python Built-in Exceptions
11. Python Language Reference
13. Python Compound Statements (with, if, for, while)
15. collections — Container datatypes
17. functools — Higher-order functions
19. dataclasses — Data Classes
21. abc — Abstract Base Classes
23. asyncio — Asynchronous I/O
25. asyncio Synchronization Primitives
27. threading — Thread-based parallelism
29. concurrent.futures — Launching parallel tasks
31. logging — Logging facility for Python
33. logging.config — Logging configuration
35. pathlib — Object-oriented filesystem paths
37. os — Miscellaneous operating system interfaces
39. venv — Creation of virtual environments
41. annotationlib — Introspection of annotations
43. Python HOWTOs
45. Descriptor HOWTO Guide
47. Logging HOWTO
49. Regex HOWTO
51. Writing pyproject.toml
2. Python 3.14 What's New
4. Python 3.12 What's New
6. Python 3.10 What's New
8. Python Built-in Functions
10. Python Data Model
12. Python Expressions
14. Python Simple Statements
16. itertools — Functions for iterating over iterables
18. typing — Support for type hints
20. enum — Support for enumerations
22. contextlib — Utilities for with-statement contexts
24. asyncio Tasks and Coroutines
26. asyncio Queues
28. multiprocessing — Process-based parallelism
30. concurrent.interpreters — Multiple interpreters in the same process
32. logging.handlers — Logging handlers
34. re — Regular expression operations
36. io — Core tools for working with streams
38. sys — System-specific parameters and functions
40. pip documentation
42. Python Glossary
44. Functional Programming HOWTO
46. Sorting HOWTO
48. Logging Cookbook
50. Python Packaging User Guide
52. Python Type System Reference

PEPs (Python Enhancement Proposals)

53. PEP 8 — Style Guide for Python Code
55. PEP 257 — Docstring Conventions
57. PEP 526 — Syntax for Variable Annotations
59. PEP 557 — Data Classes
61. PEP 585 — Type Hinting Generics in Standard Collections
63. PEP 634 — Structural Pattern Matching
65. PEP 636 — Structural Pattern Matching Tutorial
67. PEP 649 — Deferred Evaluation of Annotations
69. PEP 698 — Override decorator for static typing
71. PEP 719 — Python 3.13 Release Schedule
73. PEP 734 — Multiple Interpreters in the stdlib
75. PEP 750 — Template strings (t-strings)
77. PEP 3107 — Function Annotations
54. PEP 20 — The Zen of Python
56. PEP 484 — Type Hints
58. PEP 544 — Protocols: Structural subtyping
60. PEP 572 — Assignment Expressions (Walrus Operator)
62. PEP 604 — Allow X | Y syntax for unions
64. PEP 635 — Structural Pattern Matching Motivation
66. PEP 646 — Variadic Generics
68. PEP 695 — Type Parameter Syntax
70. PEP 703 — Making the Global Interpreter Lock Optional
72. PEP 729 — Typing governance process
74. PEP 749 — Implementing PEP 649 (Deferred Annotations)
76. PEP 779 — Criteria for supported status for free-threaded Python

Real Python Tutorials

78. Real Python — Python Type Checking Guide
80. Real Python — Primer on Python Decorators
82. Real Python — Async IO in Python
84. Real Python — Speed Up Python with Concurrency
86. Real Python — Regular Expressions in Python
88. Real Python — Python Generators Guide
90. Real Python — Python Enum Guide
92. Real Python — Python functools Module
94. Real Python — Python Scope and LEGB Rule
96. Real Python — Structural Pattern Matching in Python 3.10
98. Real Python — Python Walrus Operator
100. Real Python — Python 3.14 Preview
79. Real Python — Python Data Classes
81. Real Python — Python Metaclasses
83. Real Python — Python Concurrency Guide
85. Real Python — Python Virtual Environments Guide
87. Real Python — Python Logging Guide
89. Real Python — Python Itertools Guide
91. Real Python — Python collections Module
93. Real Python — Python Context Managers
95. Real Python — Python String Formatting Best Practices
97. Real Python — Python 3.12 f-strings
99. Real Python — Python Annotations

Technical Blogs & Tutorials

- 101. Astral uv Documentation
- 103. attrs Documentation
- 105. mypy Documentation
- 107. poetry Documentation
- 109. Ben Hoyt — Pattern Matching in Python 3.10
- 111. OpenSourceForu — Python 3.13 and 3.14 Features
- 113. Python Speed — Profiling Python Programs
- 115. tonybaloney.github.io — Python 3.13 free-threading
- 117. Brett Cannon — How importlib.resources works
- 119. Effective Python 3rd Edition (Addison-Wesley)
- 121. Python 3 Patterns, Recipes and Idioms (Bruce Eckel)
- 123. Cloudflare Blog — Python async patterns
- 125. Corey Schafer Python Tutorials
- 127. Arjan Codes — Python Design Patterns
- 102. uv CLI Reference
- 104. Pydantic Documentation
- 106. pyright Documentation
- 108. pyenv on GitHub
- 110. Cloudsmith — Python 3.14 Changes
- 112. Hynek Schlawack — Structlog (Python logging)
- 114. Python Tips — Generators and Itertools
- 116. Itamar Turner-Trauring — Getting full value from Python type hints
- 118. Fluent Python 2nd Edition (O'Reilly)
- 120. Python Tricks: The Book (Dan Bader)
- 122. Twilio Blog — Python Concurrency Comparison
- 124. Python Engineer — Python 3.12 new features
- 126. Martin Heinz — Python Project Setup in 2024
- 128. Python Discord Community

GitHub Repositories & Code Examples

- 129. CPython source code (GitHub)
- 131. mypy (GitHub)
- 133. attrs (GitHub)
- 135. astral-sh/ruff (GitHub)
- 137. poetry (GitHub)
- 139. pendulum (GitHub)
- 141. httpx (GitHub)
- 143. Python type_extensions (GitHub)
- 145. Python patterns (GitHub)
- 147. 30-seconds-of-python (GitHub)
- 130. Python Type Stubs (typeshed)
- 132. pydantic (GitHub)
- 134. astral-sh/uv (GitHub)
- 136. pyenv (GitHub)
- 138. structlog (GitHub)
- 140. Rich — terminal output library (GitHub)
- 142. aiohttp (GitHub)
- 144. awesome-python (GitHub)
- 146. wtfpython — Python quirks explained (GitHub)

Video Resources

- 148. mCoding — Python Concurrency (YouTube)
- 150. Corey Schafer — Python Decorators (YouTube)
- 152. Arjan Codes — Python Design Patterns (YouTube)
- 154. PyCon US 2025 (YouTube)
- 156. Python Bytes Podcast
- 158. David Beazley — Python Concurrency from the Ground Up (PyCon 2015)
- 160. James Powell — So you want to be a Python expert (PyData 2017)
- 149. mCoding — Python Type Hints (YouTube)
- 151. Corey Schafer — Python OOP Tutorials (YouTube)
- 153. PyCon US 2024 Talks (YouTube)
- 155. Talk Python to Me Podcast
- 157. Raymond Hettinger — Beyond PEP 8 (PyCon 2015)
- 159. Brett Slatkin — Refactoring Python (PyCon 2016)
- 161. Ned Batchelder — Facts and Myths about Python names and values

Industry Best Practice Guides & Books

- 162. Google Python Style Guide
- 164. Black Code Formatter
- 166. Ruff — Fast Python linter
- 168. Safety — Dependency security scanner
- 170. Clean Code in Python (Mariano Anaya)
- 172. High Performance Python 2nd Edition (O'Reilly)
- 174. Python Cookbook 3rd Edition (O'Reilly)
- 176. pytest Documentation
- 178. coverage.py Documentation
- 180. OWASP Python Security
- 163. Python Best Practices for Code Quality (Wemake)
- 165. isort — Import sorting
- 167. Bandit — Security linter for Python
- 169. Python Testing with pytest (Brian Okken)
- 171. Architecture Patterns with Python (Harry Percival & Bob Gregory)
- 173. CPython Internals (Anthony Shaw)
- 175. Python One-Liners (Mayer)
- 177. Python unittest Documentation
- 179. pre-commit hooks

Other

- 181. Python Weekly Newsletter
- 183. Import Python Newsletter
- 185. Python Package Index (PyPI)
- 187. Python Discuss (discuss.python.org)
- 189. Python subreddit
- 191. Python Wiki
- 193. Python Benchmarks (pyperformance)
- 195. Jupyter Project Documentation
- 197. Python REPL improvements (ptpython)
- 182. PyCoder's Weekly
- 184. Awesome Python — curated list of frameworks and tools
- 186. CPython issue tracker (GitHub Issues)
- 188. Stack Overflow — Python tag
- 190. learnpython subreddit
- 192. Python Speed Center
- 194. pyperf — Python benchmarking toolkit
- 196. IPython Documentation
- 198. Python Cheat Sheet Community (gto76)

199. [Devhints Python Cheat Sheet](#)

200. [Learn X in Y Minutes — Python](#)